

```

In [9]: import tensorflow as tf
import numpy as np
import random # Required for random sampling
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.datasets import mnist
from tensorflow.keras.optimizers import Adam

# Load and Normalize MNIST
(X_train, y_train), (X_test, y_test) = mnist.load_data()
X_train, X_test = X_train / 255.0, X_test / 255.0

param_dist = {
    'learning_rate': [0.01, 0.001, 0.0005],
    'units': [64, 128, 256, 512],
    'batch_size': [16, 32, 64, 128]
}

n_iter = 5
best_accuracy = 0
best_params = {}

print(f"Starting Random Search ({n_iter} iterations)...\n")

with tf.device('/GPU:0'):
    for i in range(n_iter):
        lr = random.choice(param_dist['learning_rate'])
        units = random.choice(param_dist['units'])
        batch = random.choice(param_dist['batch_size'])

        model = Sequential([
            Flatten(input_shape=(28, 28)),
            Dense(units, activation='relu'),
            Dense(10, activation='softmax')
        ])

        model.compile(
            optimizer=Adam(learning_rate=lr),
            loss='sparse_categorical_crossentropy',
            metrics=['accuracy']
        )

        model.fit(X_train, y_train, epochs=3, batch_size=batch, validation_split=
_, acc = model.evaluate(X_test, y_test, verbose=0)
        print(f"Iter {i+1}: LR={lr}, Units={units}, Batch={batch} -> Accuracy={a

        if acc > best_accuracy:
            best_accuracy = acc
            best_params = {'learning_rate': lr, 'units': units, 'batch_size': ba

print("\nBest Accuracy:", best_accuracy)
print("Best Hyperparameters:", best_params)

```

Starting Random Search (5 iterations)...

Iter 1: LR=0.01, Units=256, Batch=32 -> Accuracy=0.9585
Iter 2: LR=0.01, Units=512, Batch=16 -> Accuracy=0.9483
Iter 3: LR=0.0005, Units=512, Batch=64 -> Accuracy=0.9713
Iter 4: LR=0.01, Units=512, Batch=32 -> Accuracy=0.9571
Iter 5: LR=0.01, Units=512, Batch=16 -> Accuracy=0.9511

Best Accuracy: 0.9713000059127808

Best Hyperparameters: {'learning_rate': 0.0005, 'units': 512, 'batch_size': 64}

```
In [10]: print(f"\nTraining final model with: {best_params}")

final_model = Sequential([
    Flatten(input_shape=(28, 28)),
    Dense(best_params['units'], activation='relu'),
    Dense(10, activation='softmax')
])

final_model.compile(
    optimizer=Adam(learning_rate=best_params['learning_rate']),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

history = final_model.fit(
    X_train, y_train,
    epochs=10,
    batch_size=best_params['batch_size'],
    validation_split=0.2,
    verbose=1
)

loss, accuracy = final_model.evaluate(X_test, y_test, verbose=0)
print(f"\nFinal Test Accuracy using Best Hyperparameters: {accuracy*100:.2f}%")
```

Training final model with: {'learning_rate': 0.0005, 'units': 512, 'batch_size': 64}

Epoch 1/10
750/750  11s 12ms/step - accuracy: 0.9129 - loss: 0.3192 - val_accuracy: 0.9536 - val_loss: 0.1676

Epoch 2/10
750/750  11s 13ms/step - accuracy: 0.9604 - loss: 0.1398 - val_accuracy: 0.9632 - val_loss: 0.1225

Epoch 3/10
750/750  9s 11ms/step - accuracy: 0.9733 - loss: 0.0933 - val_accuracy: 0.9698 - val_loss: 0.1013

Epoch 4/10
750/750  12s 14ms/step - accuracy: 0.9799 - loss: 0.0692 - val_accuracy: 0.9728 - val_loss: 0.0906

Epoch 5/10
750/750  10s 14ms/step - accuracy: 0.9851 - loss: 0.0517 - val_accuracy: 0.9726 - val_loss: 0.0896

Epoch 6/10
750/750  20s 14ms/step - accuracy: 0.9890 - loss: 0.0400 - val_accuracy: 0.9745 - val_loss: 0.0802

Epoch 7/10
750/750  19s 12ms/step - accuracy: 0.9923 - loss: 0.0301 - val_accuracy: 0.9768 - val_loss: 0.0767

Epoch 8/10
750/750  10s 14ms/step - accuracy: 0.9944 - loss: 0.0234 - val_accuracy: 0.9768 - val_loss: 0.0774

Epoch 9/10
750/750  20s 12ms/step - accuracy: 0.9960 - loss: 0.0179 - val_accuracy: 0.9763 - val_loss: 0.0764

Epoch 10/10
750/750  10s 12ms/step - accuracy: 0.9970 - loss: 0.0146 - val_accuracy: 0.9760 - val_loss: 0.0809

Final Test Accuracy using Best Hyperparameters: 97.80%